

Entwurf und Implementierung eines interaktiven Interpreter-Ökosystems für eine eingeschränkte C++-Teilmenge

mit Typprüfung, REPL, Language-Server und GenAI-Assistenz

Projektbericht · Technische Dokumentation

Autoren	Clemens Vogtländer, Dennis Gorpnic
Lehrveranstaltung	Compilerbau (CB) – Erweiterung von Aufgabenblatt 08
Hochschule / Semester	[TODO: Hochschule, Studiengang, Semester, Betreuer/in]
Repository	cpp-subset-interpreter, Branch interpreter (Stand 27.09.2026)
Dokumentstatus	Entwurf v0.1 – nicht zur Abgabe bestimmt

Kurzfassung

Dieser Bericht dokumentiert Entwurf und Umsetzung von *MiniC++*, einem formal abgegrenzten Subdialekt von C++, sowie des darum gebauten Werkzeug-Ökosystems. Kern ist ein in Java 21 implementierter Tree-Walking-Interpreter mit ANTLR4-Frontend, zweiphasigem Resolver, deklarationsbasierter Typprüfung mit Überladungsauflösung und einem Laufzeitmodell für Referenzen, Zeiger, Heap-Objekte, Slicing und virtuellen Dispatch. Darauf setzen eine REPL, ein Language-Server nach LSP 3.17 mit VS-Code-Extension und ein REST-basierter GenAI-Assistenzserver (Ollama) auf, dessen Antworten durch den echten Compiler abgesichert werden. 218 automatisierte Tests, ein differenzieller Vergleich mit GCC in der CI und JMH-Benchmarks bilden die Evaluationsgrundlage.

Inhaltsverzeichnis

1 Einleitung	3
1.1 Ausgangslage und Zielsetzung	3
1.2 Projektumfang	3
1.3 Aufbau des Dokuments	3
2 Die Sprache MiniC++	4
2.1 Typen, Variablen und Werte	4
2.2 Ausdrücke und Operatoren	4
2.3 Anweisungen, Funktionen und Überladung	5
2.4 Klassen, Vererbung und Polymorphie	5
2.5 Referenzen und Zeiger	5
2.6 Bewusste Einschränkungen und Präzisierungen	6
3 Systemarchitektur	6
3.1 Modulstruktur	6
3.2 Verarbeitungspipeline	7
3.3 Die Fassade MiniCpp	7
3.4 Architekturentscheidungen	8
4 Kern-Interpreter	9
4.1 Lexikalische Analyse	9
4.2 Syntaxanalyse	9
4.3 Abstrakter Syntaxbaum	10
4.4 Namensauflösung	10
4.5 Typprüfung	11
4.6 Laufzeitmodell	11
4.7 REPL	13
4.8 Kommandozeile und C++-Export	13
5 Language-Server	14
5.1 Architektur	14
5.2 Synchronisation und Entprellung	14
5.3 Analyse-Snapshot und Fehlertoleranz	15
5.4 Symbolindex	15
5.5 Features	16
5.6 VS-Code-Extension	16
6 GenAI-Assistenzserver	16
6.1 Endpunkte	17
6.2 Provider-Abstraktion	17
6.3 Prompt-Design	17
6.4 Absicherung durch den Compiler	17
7 Qualitätssicherung und Evaluation	18
7.1 Teststrategie	18
7.2 Differenzieller Vergleich mit GCC	19
7.3 Performance und Skalierbarkeit	19
7.4 Bewertung	20
8 Projektorganisation	21
8.1 Arbeitsplan	21
8.2 Aufgabenteilung	21
9 Grenzen und Ausblick	22
9.1 Bekannte Grenzen	22
9.2 Mögliche Weiterentwicklungen	22
9.3 Fazit	22
Anhang	23

1 Einleitung

1.1 Ausgangslage und Zielsetzung

Aufgabenblatt 08 der Lehrveranstaltung Compilerbau verlangt einen Interpreter für eine Teilmenge von C++ mit Klassen, Einfachvererbung, Referenzen und virtuellen Methoden. Das hier beschriebene Projekt nimmt diese Aufgabe als Kern und erweitert sie zu einem vollständigen *Ökosystem*: Neben der reinen Ausführung von Programmen soll die Sprache so benutzbar sein, wie man es von einer produktiven Programmiersprache erwartet – mit einer interaktiven Shell, Editor-Unterstützung und KI-gestützter Code-Assistenz.

Daraus ergeben sich drei übergeordnete Ziele:

- **Korrektheit gegenüber C++.** Jedes gültige MiniC++-Programm soll sich genau so verhalten wie das entsprechende Standard-C++-Programm. Wo C++ undefiniertes Verhalten zulässt (hängende Zeiger, doppeltes delete), soll MiniC++ stattdessen einen präzisen Laufzeitfehler melden.
- **Wiederverwendbarkeit des Frontends.** Lexer, Parser, Resolver und Typprüfung sollen nicht nur dem Interpreter dienen, sondern über eine schmale Fassade auch dem Language-Server und dem GenAI-Server.
- **Messbarkeit.** Korrektheit, Performance und Skalierbarkeit sollen mit reproduzierbaren Werkzeugen (JUnit, GCC-Vergleich, JMH) belegt werden.

1.2 Projektumfang

Das System gliedert sich in vier Säulen, die in diesem Bericht jeweils ein eigenes Kapitel erhalten:

Säule	Inhalt	Verantwortung
Kern-Interpreter	Lexer, Parser, AST, Resolver, Typprüfung, Tree-Walking-Interpreter, REPL, CLI, C++-Export	gemeinsam
LSP-Server	Language Server Protocol 3.17 über LSP4J, VS-Code-Extension	Clemens Vogtländer
GenAI-Server	REST-Endpunkte für Vervollständigung, Erklärung, Refactoring und Fehlersuche mit Ollama	Dennis Gorpinic
Evaluation	Unit- und Golden-File-Tests, differenzieller GCC-Vergleich, JMH-Benchmarks	gemeinsam

Tabelle 1: Die vier Säulen des Projekts

1.3 Aufbau des Dokuments

Kapitel 2 definiert den Sprachumfang von MiniC++ einschließlich aller Präzisierungen, die über die Aufgabenstellung hinaus getroffen wurden. Kapitel 3 beschreibt die Gesamtarchitektur und die Architekturentscheidungen. Kapitel 4 behandelt den Kern-Interpreter Phase für Phase, Kapitel 5 den Language-Server und Kapitel 6 den GenAI-Assistenzserver. Kapitel 7 fasst Teststrategie und Messergebnisse zusammen, Kapitel 8 die Projektorganisation. Kapitel 9 diskutiert Grenzen und mögliche Weiterentwicklungen. Der Anhang enthält eine Kurzreferenz der Kommandozeile und die Kernregeln der Grammatik.

Konventionen: Bezeichner aus dem Quellcode sind in Festbreitenschrift gesetzt; Pfade beziehen sich auf die Wurzel des Repositorys. Das Java-Basispaket `de.cvogtlaender` wird in Klassennamen weggelassen.

2 Die Sprache MiniC++

MiniC++ ist eine echte Teilmenge von C++17: Jedes gültige MiniC++-Programm ist – nach einer rein mechanischen Übersetzung, siehe Abschnitt 4.8 – ein gültiges C++-Programm mit identischer Ausgabe. Dieses Kapitel beschreibt den Sprachumfang aus Sicht des Programmierers.

2.1 Typen, Variablen und Werte

MiniC++ kennt fünf Basistypen, benutzerdefinierte Klassentypen sowie daraus gebildete Referenz- und Zeigertypen. Variablen werden mit `T x;` oder `T x = expr;` deklariert; für Klassentypen ist `T x(a, b);` eine Kurzform von `T x = T(a, b);`. Globale Variablen gibt es nicht.

Typ	Wertebereich / Semantik	Standardwert ohne Initialisierer
<code>int</code>	32 Bit, Zweierkomplement mit Überlauf (wie <code>g++ -fwrapv</code>)	<code>0</code>
<code>bool</code>	<code>true</code> / <code>false</code>	<code>false</code>
<code>char</code>	Zeichen inkl. Escape-Sequenzen <code>\n</code> <code>\t</code> <code>\'</code> <code>\\</code> <code>\0</code> u. a.	<code>'\0'</code>
<code>string</code>	Zeichenkette mit Wertsemantik	<code>""</code>
<code>void</code>	nur als Rückgabotyp	<code>–</code>
Klasse A	Objekt mit Wertsemantik (Kopie bei Zuweisung)	parameterloser Konstruktor
<code>T&</code>	Referenz; Initialisierung obligatorisch, keine Neubindung	<code>–</code> (Fehler)
<code>T*</code> , <code>T**</code>	Zeiger, auch mehrstufig; keine Arithmetik	<code>nullptr</code>

Tabelle 2: Typen von MiniC++ und ihre Standardwerte

2.2 Ausdrücke und Operatoren

Operatorpräzedenz und Assoziativität folgen dem C++-Standard. Die Typregeln sind bewusst strenger als in C++: Es gibt keine impliziten arithmetischen Konvertierungen, und beide Operanden eines binären Operators müssen denselben Typ haben.

Stufe	Operatoren	zulässige Operandentypen
1 (höchste)	Aufruf <code>f()</code> , Member <code>.</code> <code>-></code>	Funktionen, Methoden, Objekte, Zeiger auf Objekte
2	unär <code>!</code> <code>+</code> <code>-</code> <code>*</code> <code>&</code> , <code>new</code>	<code>!</code> nur <code>bool</code> ; <code>+</code> <code>-</code> nur <code>int</code> ; <code>*</code> Zeiger; <code>&</code> L-Werte
3	<code>*</code> <code>/</code> <code>%</code>	<code>int</code>
4	<code>+</code> <code>-</code>	<code>int</code>
5	<code><</code> <code><=</code> <code>></code> <code>>=</code>	<code>int</code> , <code>char</code>
6	<code>==</code> <code>!=</code>	<code>int</code> , <code>char</code> , <code>bool</code> , <code>string</code> , kompatible Zeiger
7	<code>&&</code>	<code>bool</code> (Kurzschlussauswertung)
8	<code> </code>	<code>bool</code> (Kurzschlussauswertung)
9 (niedrigste)	<code>=</code> (rechtsassoziativ)	L-Wert links; Slicing bei Klassen erlaubt

Tabelle 3: Operatoren nach Präzedenz

Implizite Konvertierung nach `bool` findet ausschließlich in `if`- und `while`-Bedingungen statt; dort sind auch `int`, `char` und Zeiger erlaubt. Operanden und Argumente werden strikt von links nach rechts ausgewertet. Division und Modulo durch null sind Laufzeitfehler.

2.3 Anweisungen, Funktionen und Überladung

Der Kontrollfluss umfasst Blöcke, `if/else`, `while` und `return`; `for`, `switch`, `break` und `continue` gehören nicht zur Sprache. Funktionen können über Name, Arität und Parametertypen einschließlich der `&`-Markierung überladen werden. Eingebaut sind `print_bool`, `print_int`, `print_char` und `print_string`, die ihren Wert gefolgt von einem Zeilenumbruch ausgeben. Einstiegspunkt ist `int main()` oder `void main()`; `int main()` ohne `return` liefert 0, und der Rückgabewert wird zum Exit-Code des Prozesses.

2.4 Klassen, Vererbung und Polymorphie

Klassen werden als `class A { public: ... };` definiert; alle Member sind öffentlich. Fehlt ein Konstruktor, wird ein parameterloser synthetisiert. Einfachvererbung erfolgt über `class D : public B`, wobei vor dem Rumpf eines Konstruktors von D implizit der parameterlose Konstruktor von B läuft. Die Namensauflösung innerhalb von Methoden folgt der Reihenfolge *lokal* → *eigene Member* → *geerbte Member* → *global*; wie in C++ verdecken Member einer abgeleiteten Klasse gleichnamige Member der Basisklasse. `this` existiert nicht.

Polymorphie entsteht über `virtual`-Methoden, die durch Referenzen oder Zeiger aufgerufen werden. Eine Methode, die eine virtuelle Methode überschreibt, ist selbst virtuell. Während der Konstruktor einer Basisklasse läuft, wird – exakt wie in C++ – deren eigene Version aufgerufen. Die Zuweisung eines abgeleiteten Objekts an eine Variable vom Basistyp kopiert nur den Basisanteil (*Slicing*).

```
class Account {
public:
    int balance;
    virtual int monthlyFee() { return 5; }
    void endOfMonth() { balance = balance - monthlyFee(); } // dynamischer Dispatch
};
class StudentAccount : public Account {
public:
    int monthlyFee() { return 0; } // implizit virtual
};
int main() {
    StudentAccount bob;
    Account& any = bob;    print_int(any.monthlyFee()); // 0 (Polymorphie)
    Account copy = bob;   print_int(copy.monthlyFee()); // 5 (Slicing)
    Account* p = new StudentAccount();
    print_int(p->monthlyFee()); // 0
    delete p;
    return 0;
}
```

Listing 1: Vererbung, virtueller Dispatch und Slicing (gekürzt aus `examples/features.cpp`)

2.5 Referenzen und Zeiger

Referenzen (`T& r = x;`, Parameter `T& p`) sind zweite Namen für einen existierenden Speicherort; eine Zuweisung an `r` schreibt in `x`. Referenzfelder, Referenz-Rückgaben und nicht initialisierte Referenzen sind nicht erlaubt. Zeiger (`T*`, `T**`, auch als Parameter `T*&`, Feld oder Rückgabewert) unterstützen den Adressoperator `&x`, die Dereferenzierung `*p` (lesend und schreibend), Memberzugriff `p->m`, das Literal `nullptr`, Heap-Allokation mit `new T` bzw. `new T(args)` und Freigabe mit `delete p`; Implizit konvertiert werden nur `D* → B*` und `nullptr → T*`.

Eine Besonderheit, die direkt aus der C++-Grammatik folgt: Die Anweisung `a * b;` ist eine Deklaration eines Zeigers `b` vom Typ `a*` und keine Multiplikation.

2.6 Bewusste Einschränkungen und Präzisierungen

Nicht unterstützt werden Zeigerarithmetik, `void*`, Casts, Arrays, `++/--`, zusammengesetzte Zuweisungen, `break/continue`, Mehrfachvererbung, Templates, `static`, `const`, `this`, globale Variablen, Initialisierungslisten, Destruktoren und reine Funktionsdeklarationen. Wo die Aufgabenstellung offen war, wurden folgende Festlegungen getroffen:

- Was in C++ undefiniertes Verhalten wäre, ist ein **Laufzeitfehler**: Dereferenzieren von `nullptr`, hängende Zeiger und Referenzen auf Variablen, deren Gültigkeitsbereich endete, Zugriff auf gelöschte Heap-Objekte, doppeltes `delete` sowie `delete` auf nicht per `new` erzeugte Objekte. Nicht freigegebener Speicher ist dagegen kein Fehler.
- Mehr als 100 000 verschachtelte Aufrufe gelten als Stapelüberlauf und werden als Laufzeitfehler gemeldet.
- Überladungsauflösung: Zuerst werden exakte Treffer gesucht; nur wenn keiner existiert, werden die Konvertierungen `Derived → Base`, `D* → B*` und `nullptr → T*` berücksichtigt. Mehr als ein bester Kandidat ist ein Fehler.
- Felder dürfen in abgeleiteten Klassen nicht erneut deklariert werden.
- Funktionen und Klassen dürfen vor ihrer Definition verwendet werden (*define-after-use*), Variablen nicht.

3 Systemarchitektur

3.1 Modulstruktur

Das Repository ist ein Gradle-Multiprojekt mit vier Java-Modulen und einem TypeScript-Projekt. Alle Werkzeuge hängen ausschließlich vom Modul `interpreter` ab; untereinander gibt es keine Abhängigkeiten. Damit bleibt der Kern frei von LSP-, HTTP- oder Benchmark-Bibliotheken.

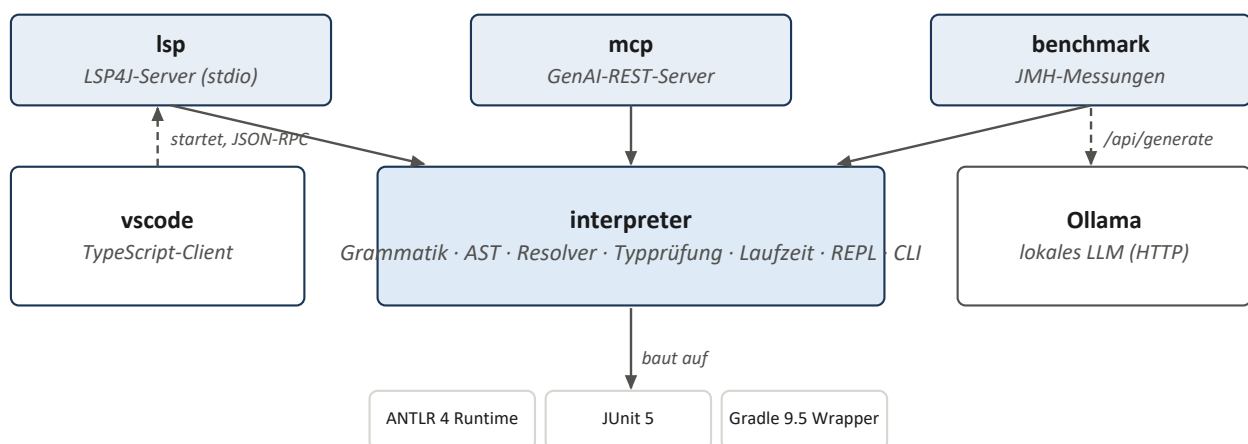


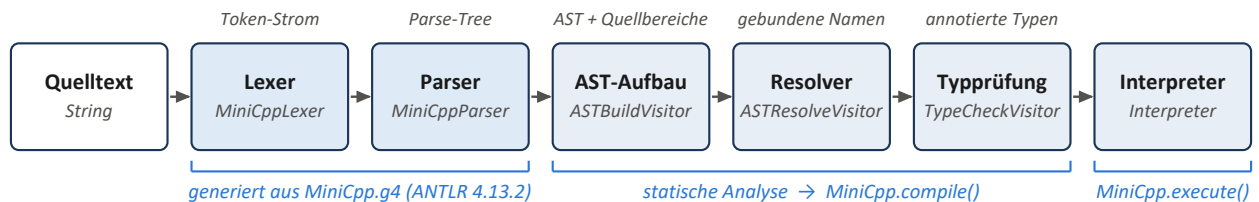
Abbildung 1: Module und ihre Abhängigkeiten

Modul	Aufgabe	Quellzeilen (main / test)	Tests
interpreter	Sprache, Laufzeit, REPL, CLI, C++-Export	6 421 / 1 680	175
lsp	Language-Server (LSP4J 1.0.0)	2 295 / 590	22
mcp	GenAI-REST-Server (JDK-HttpServer, Gson)	618 / 363	21
benchmark	JMH-Benchmarks (JMH 1.37)	262 / –	–
vscode	VS-Code-Client (TypeScript)	86	–

Tabelle 4: Module des Repositorys (Stand 27.09.2026, inkl. Grammatik und Testprogramme)

3.2 Verarbeitungspipeline

Die Verarbeitung eines Programms folgt der klassischen Phasenstruktur eines Compilers. Jede Phase ist als eigene Klasse realisiert; ab dem AST sind alle Phasen Implementierungen der generischen Schnittstelle `AstVisitor<T>`. Eine Phase läuft nur, wenn die vorherige fehlerfrei war – so kann sich jede Phase auf die Invarianten der vorherigen verlassen (etwa: der Interpreter findet jeden Bezeichner bereits aufgelöst und jeden Ausdruck typisiert vor).



Nutzer der Fassade:



Abbildung 2: Verarbeitungspipeline und Fassade MiniCpp

3.3 Die Fassade MiniCpp

Alle Nutzer greifen über die Klasse `interpreter.MiniCpp` auf die Pipeline zu. Die Ergebnisse sind als unveränderliche Java-Records modelliert, die neben dem eigentlichen Ergebnis stets eine Liste von Diagnosen tragen:

```

record ParseResult<T>(T tree, List<Diagnostic> diagnostics, boolean incomplete)
record Compilation(Program program, GlobalScope globals, List<Diagnostic> diagnostics)
record RunResult(int exitCode, List<Diagnostic> diagnostics)

static ParseResult<Program> parseProgram(String source)
static ParseResult<ReplInput> parseReplInput(String source)
static Compilation compile(String source) // Parsen + Resolver + Typprüfung
static Compilation analyze(Program p, boolean requireMain)
static RunResult run(String source, PrintStream out)
static RunResult execute(Compilation c, PrintStream out)
  
```

Listing 2: Öffentliche Schnittstelle der Fassade MiniCpp (vereinfacht)

Eine `Diagnostic` besteht aus der Phase (SYNTAX, RESOLVE, TYPE, RUNTIME), einem Quellbereich (Zeilen 1-basiert, Spalten 0-basiert, Ende exklusiv) und einer Meldung. Die Kommandozeile formatiert sie im von GCC und Clang bekannten Format `datei.cpp:3:7: error: use of undeclared identifier 'y';`; der Language-Server rechnet sie in LSP-Positionen um. Die Trennung von Ergebnis und Darstellung ist der

Grund, weshalb alle drei Werkzeuge dieselben Fehlermeldungen zeigen.

3.4 Architekturentscheidungen

Die grundlegenden Entscheidungen wurden zu Projektbeginn gemeinsam getroffen und als Architecture Decision Records festgehalten. Die folgende Tabelle fasst sie mit ihren Konsequenzen zusammen.

Entscheidung	Begründung	Konsequenz im Projekt
Java 21 als Implementierungssprache	ausgereifte Standardbibliothek, Klassenhierarchien und Visitor-Pattern passen zur AST-Modellierung; Records und Pattern-Matching verkürzen den Code	tiefe Rekursion des Tree-Walkers erfordert einen eigenen Thread mit 512 MiB Stack (Abschnitt 4.6)
ANTLR4 als Parsergenerator	De-facto-Standard auf der JVM, Lexer und Parser aus einer <code>.g4</code> -Datei, ALL(*)-Parsing, eingebaute Fehlerbehandlung	Parse-Tree muss in einen eigenen AST übersetzt werden; Grammatik wird beim Build generiert
Handgeschriebener AST statt Parse-Tree	Phasen sollen auf einer stabilen, typisierten Struktur arbeiten und Annotationen (Bindung, Typ) speichern	zusätzliche Klasse <code>ASTBuildVisitor</code> (757 Zeilen)
VS Code als LSP-Zielformat	gut dokumentierte Extension-API, <code>vscode-languageclient</code>	Server bleibt editorneutral (stdio, LSP4J)
Ollama als GenAI-Provider	lokal, ohne API-Schlüssel, Kosten oder Datenschutzbedenken	Provider-Schnittstelle mit Mock für deterministische Tests

Tabelle 5: Architekturentscheidungen

Komponente	Version
JDK (Ziel / Entwicklung)	21 / 25
Gradle (Wrapper)	9.5.1
ANTLR	4.13.2
Eclipse LSP4J	1.0.0
Gson	2.11.0
JUnit Jupiter	5.11.4
JMH	1.37
vscode-languageclient / VS Code Engine	10.1 / ≥ 1.91
CI	GitHub Actions, Temurin 21, Node 22

Tabelle 6: Technologie-Stack

4 Kern-Interpreter

Dieses Kapitel beschreibt die Phasen des Kern-Interpreters in der Reihenfolge, in der ein Programm sie durchläuft. Der Schwerpunkt liegt auf den Entwurfsentscheidungen, die nicht unmittelbar aus dem Code ersichtlich sind.

4.1 Lexikalische Analyse

Der Lexer wird aus dem Lexer-Teil von `interpreter/src/main/antlr4/MiniCpp.g4` generiert. Schlüsselwörter (einschließlich `new`, `delete` und `nullptr`) sind eigene Token-Typen und stehen vor der Identifizierungsregel, sodass ANTLR bei gleicher Länge das Schlüsselwort bevorzugt. Leerraum, Zeilen- und Blockkommentare sowie Präprozessorzeilen (`#include <iostream>`) werden verworfen – so lassen sich MiniC++-Programme unverändert auch mit einem C++-Compiler übersetzen. Zeichen- und Zeichenkettenlitterale erlauben die Escape-Sequenzen `\b \t \n \r \" \' \\ \0`. Jedes Token trägt Zeile und Spalte; diese Positionen wandern über den AST bis in die Diagnosen.

4.2 Syntaxanalyse

Der Parser besitzt zwei Startregeln: `program` für Dateien (nur Deklarationen) und `replInput` für die REPL, die Deklarationen und Anweisungen in beliebiger Reihenfolge und einen abschließenden Ausdruck ohne Semikolon erlaubt. Die Ausdrucksgrammatik ist als Präzedenzkaskade formuliert – eine Regel pro Präzedenzstufe –, was die Präzedenz explizit und die Grammatik frei von Mehrdeutigkeiten hält:

```
assignmentExpr    : logicalOrExpr (ASSIGN assignmentExpr)? ;           // rechtsassoziativ
logicalOrExpr     : logicalAndExpr (OROR logicalAndExpr)* ;
...
multiplicativeExpr : unaryExpr ((STAR | SLASH | PERCENT) unaryExpr)* ;
unaryExpr         : (NOT | PLUS | MINUS | STAR | AMP) unaryExpr | newExpr | postfixExpr ;
postfixExpr       : primaryExpr postfixPart* ;
postfixPart       : LPAREN argList? RPAREN | DOT Identifier | ARROW Identifier ;

type              : (baseType | Identifier) STAR* ;                   // 'T', 'T*', 'T**'
statement         : block | varDecl SEMI | ifStmt | whileStmt
                  | returnStmt SEMI | deleteStmt SEMI | expr SEMI ;
```

Listing 3: Auszug aus der Parser-Grammatik

Da `varDecl` in `statement` vor `expr` steht und ANTLR bei Mehrdeutigkeit die erste passende Alternative wählt, wird `a * b;` – wie in C++ – als Zeigerdeklaration erkannt. Die Grammatik akzeptiert bewusst etwas mehr als die Sprache (z. B. Klassenbezeichner an beliebiger Typposition); die Einschränkungen prüfen Resolver und Typprüfung, die präzisere Fehlermeldungen liefern können als der Parser.

Fehlerbehandlung

Die Standard-Fehlerausgabe von ANTLR wird durch einen `SyntaxErrorCollector` ersetzt, der jeden Fehler als `Diagnostic` mit der Länge des betroffenen Tokens aufzeichnet. Zusätzlich merkt er sich, ob ein Fehler *am Dateiende* auftrat. Dieses Flag (`ParseResult.incomplete`) nutzt die REPL, um unvollständige Eingaben – etwa eine offene geschweifte Klammer – von echten Syntaxfehlern zu unterscheiden. Bei Syntaxfehlern wird kein AST erzeugt; die Fehlerbehandlung von ANTLR sorgt aber dafür, dass alle Syntaxfehler einer Datei in einem Durchlauf gemeldet werden.

4.3 Abstrakter Syntaxbaum

Der `ASTBuildVisitor` übersetzt den Parse-Tree in einen handgeschriebenen AST. Dabei werden syntaktische Details entfernt (Klammern, Präzedenzstufen), Kurzformen normalisiert (`T x(a)` wird zu einer Deklaration mit Konstruktoraufwurf) und jedem Knoten sein vollständiger Quellbereich mitgegeben. Die Knoten gliedern sich in vier Hierarchien:

Basisklasse	Knoten
<code>Decl</code>	<code>ClassDecl</code> , <code>ConstructorDecl</code> , <code>FieldDecl</code> , <code>FunctionDecl</code> , <code>MethodDecl</code> , <code>ParameterDecl</code> , <code>VariableDecl</code>
<code>Stmt</code>	<code>BlockStmt</code> , <code>VariableStmt</code> , <code>ExprStmt</code> , <code>IfStmt</code> , <code>WhileStmt</code> , <code>ReturnStmt</code> , <code>DeleteStmt</code>
<code>Expr</code>	<code>AssignExpr</code> , <code>BinaryExpr</code> , <code>UnaryExpr</code> , <code>CallExpr</code> , <code>MemberAccessExpr</code> , <code>NewExpr</code> , <code>VarExpr</code> , Literale (<code>Int</code> , <code>Bool</code> , <code>Char</code> , <code>String</code> , <code>Nullptr</code>), <code>ErrorExpr</code>
<code>Type</code>	<code>PrimitiveType</code> , <code>ClassType</code> , <code>PointerType</code> , <code>ReferenceType</code> , <code>NullptrType</code>

Tabelle 7: Knotenhierarchien des AST

Wurzelknoten sind `Program` (Datei) und `ReplInput` (REPL-Eingabe). Spätere Phasen annotieren den AST, statt eigene Tabellen aufzubauen: Der Resolver setzt an `VarExpr` die referenzierte Deklaration, die Typprüfung setzt an jedem `Expr` den inferierten Typ und an `CallExpr` bzw. `NewExpr` die gewählte Überladung. Genau diese Annotationen nutzt der Language-Server später für Hover, Go-to-Definition und Umbenennen.

4.4 Namensauflösung

Der `ASTResolveVisitor` arbeitet in zwei Durchläufen. **Pass 1** sammelt alle Klassen und Funktionen im `GlobalScope`, verknüpft jede Klasse mit ihrer Basisklasse, erkennt zyklische Vererbung und prüft Member-Deklarationen (doppelte Felder, erneut deklarierte Felder in abgeleiteten Klassen, doppelte Signaturen). Weil alle globalen Namen danach bekannt sind, dürfen Funktionen und Klassen vor ihrer Definition verwendet werden. **Pass 2** durchläuft alle Rümpfe mit einem Stapel lokaler Gültigkeitsbereiche und bindet jeden Bezeichner in der Reihenfolge *lokal* → *eigene Member* → *geerbte Member* → *global*. Da lokale Namen erst beim Erreichen ihrer Deklaration in den aktuellen Bereich eingetragen werden, entsteht für Variablen automatisch die *define-before-use*-Semantik.

```
int main() { x = 1; int x; }           // 1:14: error: use of undeclared identifier 'x'
int main() { int x; bool x; }         // error: redeclaration of 'x'
class A : public B { public: };       // error: unknown base class 'B'
class A : public B { public: }; class B : public A { public: }; // error: 'A' inherits from itself
void f(int a) { } void f(int b) { }   // error: redefinition of function 'f(int)'
```

Listing 4: Typische Fehler der Namensauflösung (aus `ErrorTest`)

Für die REPL stellt der Resolver zusätzlich Einzeloperationen bereit (`declareClass`, `declareFunction`, `resolveSessionStatement`), mit denen Eingaben einzeln und damit stets *define-before-use* aufgelöst werden.

4.5 Typprüfung

Da auto nicht zum Sprachumfang gehört, ist keine Typinferenz im Sinne von Hindley-Milner nötig; jede Variable trägt ihren deklarierten Typ. Der `TypeCheckVisitor` bestimmt die Typen von Ausdrücken bottom-up und prüft dabei die Regeln aus Kapitel 2: gleiche Operandentypen, zulässige Operatoren pro Typ, L-Werte links von `=` und bei Referenzparametern, Bedingungstypen, Zeigerkompatibilität und Konstruktoraufrufe. Darüber hinaus prüft er:

- **Return-Pfade:** Eine Nicht-void-Funktion muss auf allen Pfaden ein `return` erreichen (`alwaysReturns` analysiert `if/else`-Verzweigungen und Blöcke; `int main()` ist ausgenommen).
- **Overrides:** Eine Methode mit gleicher Signatur wie eine virtuelle Basismethode muss denselben Rückgabetyt haben und wird selbst als virtuell markiert (`isEffectivelyVirtual`).
- **Rekursive Einbettung:** Eine Klasse darf sich nicht selbst als Wertfeld enthalten, auch nicht indirekt (Zeigerfelder sind erlaubt).
- **Einstiegspunkt:** Es muss genau ein parameterloses `main` mit Rückgabetyt `int` oder `void` existieren.

Überladungsauflösung

Die Auflösung von Funktions-, Methoden- und Konstruktoraufrufen erfolgt zentral in `pickOverload`. Für jeden Kandidaten berechnet `matchCost` die Anzahl der nötigen impliziten Konvertierungen (`-1` = nicht aufrufbar). Ein Referenzparameter verlangt ein L-Wert-Argument und darf nicht an einen konvertierten Zeiger binden, da dieser ein temporärer Wert wäre.

```
List<D> exact = ..., converting = ...;
for (D candidate : candidates) {
    int cost = matchCost(params.apply(candidate), args, argTypes);
    if (cost == 0) exact.add(candidate);
    else if (cost > 0) converting.add(candidate);
}
List<D> best = !exact.isEmpty() ? exact : converting;
if (best.size() == 1) return best.get(0);
// sonst: "no matching function for call to 'f(int, bool)'; candidates are: ..."
// bzw. "call to 'f(B*)' is ambiguous; candidates are: ..."
```

Listing 5: Kern der Überladungsauflösung in `TypeCheckVisitor`

4.6 Laufzeitmodell

Der Interpreter ist ein Tree-Walking-Interpreter: Er implementiert `AstVisitor<Object>`, wobei Ausdrücke zu Laufzeitwerten und Anweisungen zu `null` oder einem `Returned-Signal` auswerten. Die Rückgabe über ein Signalobjekt statt über eine Java-Exception hält den Aufrufpfad frei von teurem Stack-Unwinding. Laufzeitwerte sind `Integer`, `Boolean`, `Character`, `String`, `Pointer` oder `ObjectValue`.

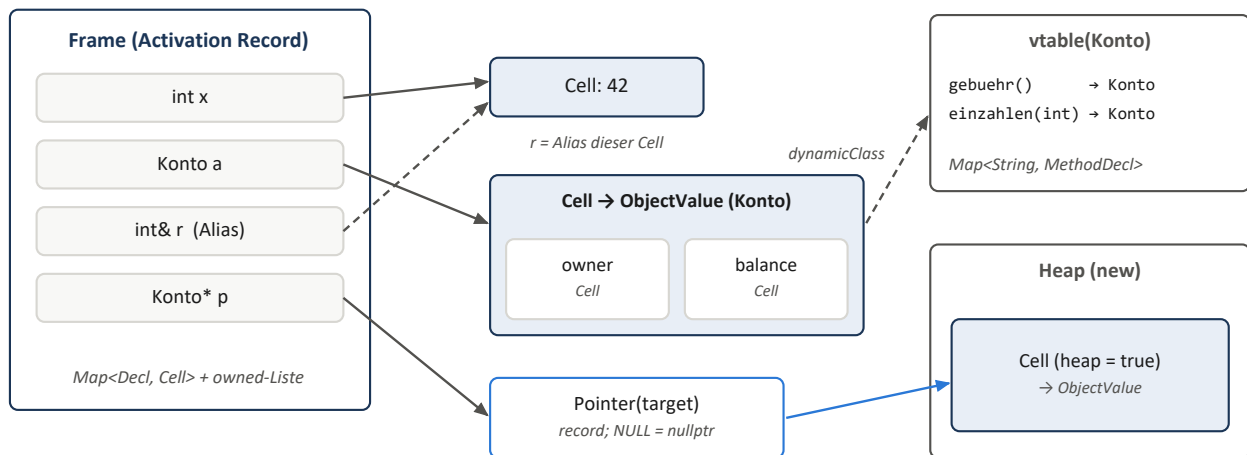


Abbildung 3: Speichermodell: Frames, Cells, Objekte, Zeiger und vtables

Cells als Speicherorte

Zentrale Abstraktion ist die *Cell*, ein veränderlicher Speicherort. Jede Variable, jeder By-Value-Parameter und jedes Feld besitzt genau eine *Cell*. Eine **Referenz** ist schlicht ein zweiter Name für dieselbe *Cell* – Zuweisungen über die Referenz ändern damit automatisch das Original. Ein **Zeiger** ist der Record `Pointer(Cell target)`; `nullptr` ist `Pointer(null)`, und zwei Zeiger sind gleich, wenn sie auf dieselbe *Cell* zeigen.

Lebensdauer und Fehlererkennung

Jeder Frame (Activation Record) führt eine Liste der *Cells*, die ihm gehören. Endet ein Block oder eine Funktion, werden diese *Cells* per `kill()` als tot markiert – rekursiv einschließlich der Felder enthaltener Objekte. Heap-*Cells* aus `new` tragen zusätzlich das Flag `heap`; `delete` prüft, dass der Zeiger auf eine lebende Heap-*Cell* zeigt, und tötet sie. Jeder Zugriff über einen Zeiger oder eine Referenz prüft den Zustand der Ziel-*Cell*. So werden `nullptr`-Dereferenzierung, hängende Zeiger, Use-after-free, doppeltes `delete` und `delete` auf Stack-Objekte zuverlässig und mit Quellposition erkannt, ohne eine Speicherverwaltung zu simulieren – die eigentliche Freigabe übernimmt der Garbage Collector der JVM.

Objekte, Kopien und Slicing

Ein *ObjectValue* hält seine Felder in einer `LinkedHashMap<String, Cell>` in Deklarationsreihenfolge. Zuweisungen und Wertparameter kopieren Objekte tief (`copy`); ist der statische Zieltyp eine Basisklasse, entsteht mit `sliceTo` eine Kopie, die nur deren Felder enthält. Temporäre Objekte (etwa Rückgabewerte) werden nicht erneut kopiert.

Virtueller Dispatch

Für jede Klasse wird bei Bedarf eine *vtable* als `Map<String, MethodDecl>` aufgebaut: Sie übernimmt die Einträge der Basisklasse und überschreibt sie mit den eigenen Methoden. Schlüssel ist der Methodenname zusammen mit den Parametertypen, sodass überladene virtuelle Methoden getrennt bleiben. Nicht-virtuelle Aufrufe werden statisch gebunden, virtuelle über die *vtable* der *dynamischen Klasse* des Empfängers. Diese ist normalerweise die Klasse des Objekts, wird aber während eines Basisklassen-Konstruktors vorübergehend auf die Basisklasse gesetzt – das bildet die C++-Regel nach, dass im Konstruktor noch nicht in abgeleitete Klassen dispatcht wird.

```
private MethodDecl dispatch(MethodDecl method, ObjectValue receiver) {
    if (!method.isEffectivelyVirtual()) return method;
    MethodDecl override = vtable(receiver.getDynamicClass()).get(vtableKey(method));
    return override != null ? override : method;
}
```

Listing 6: Dynamischer Dispatch im Interpreter

Rekursionstiefe

Ein Tree-Walker benötigt pro interpretiertem Aufruf mehrere Java-Stackframes. Damit auch tiefe Rekursion (z. B. 100 000 Ebenen) funktioniert, führt MiniC++ `onLargeStack` den Interpreter auf einem eigenen Thread mit 512 MiB Stack aus. Ein Zähler begrenzt die Aufruftiefe auf `MAX_CALL_DEPTH = 100_000`; verbleibende `StackOverflowErrors` (etwa durch extrem tief verschachtelte Ausdrücke) werden in einen regulären Laufzeitfehler übersetzt.

4.7 REPL

Die REPL (`repl.Repl`) lädt optional eine Datei, führt deren `main` im *Sitzungs-Scope* aus und hält diesen offen, sodass die lokalen Variablen von `main` anschließend zur Verfügung stehen. Jede Eingabe wird mit der Startregel `replInput` geparkt. Meldet der Parser einen Fehler am Eingabeende, zeigt die REPL den Fortsetzungsprompt `...>` und sammelt weitere Zeilen. Eine vollständige Eingabe wird als Ganzes aufgelöst, geprüft und ausgeführt; vor der Verarbeitung wird mit `GlobalScope.snapshot()` ein Sicherungspunkt angelegt, auf den bei einem Fehler zurückgesetzt wird. Eine fehlerhafte Eingabe hinterlässt damit keine halb definierten Klassen oder Funktionen.

```
minicpp> int x = 6;
minicpp> int sq(int n) {
    ...> return n * n;
    ...> }
minicpp> sq(x) + 1
37
minicpp> :vars
int x = 6
```

Listing 7: REPL-Sitzung

Neue Variablen landen im Sitzungs-Scope, neue Funktionen und Klassen im globalen Scope. Da MiniC++ keine globalen Variablen kennt, sind Sitzungsvariablen in Funktionen nicht sichtbar. Befehle: `:vars`, `:functions`, `:classes`, `:load <datei>`, `:reset`, `:cancel`, `:help`, `:quit`.

4.8 Kommandozeile und C++-Export

Die Klasse `Main` stellt die Unterbefehle `run`, `check`, `repl`, `ast` und `to-cpp` bereit (Anhang A). Der Exit-Code von `run` ist der Rückgabewert von `main`, bei Fehlern 1.

Der `CppExporter` übersetzt ein geprüftes MiniC++-Programm in Standard-C++17 mit identischem Verhalten. Er ist die Grundlage des differenziellen Tests gegen GCC (Abschnitt 7.2) und überbrückt die Stellen, an denen MiniC++ bewusst von C++ abweicht:

- `define-after-use`: Klassen werden vorwärts deklariert und nach Abhängigkeiten sortiert, Funktionen erhalten Prototypen, Methoden werden nach allen Klassen außerhalb definiert;
- Standardwerte für nicht initialisierte Variablen und Felder, `new T` wird zu `new T()`;
- Zeichenkettenlitterale erhalten den Typ `string`, die `print_*`-Funktionen werden in einem Präludium definiert, `void main()` wird gekapselt;
- Basisklassen erhalten einen virtuellen Destruktor, damit `delete` über einen Basiszeiger definiert ist.

5 Language-Server

Der Language-Server macht MiniC++ in jedem LSP-fähigen Editor nutzbar; Referenz-Client ist eine VS-Code-Extension. Er ist in Java mit Eclipse LSP4J implementiert, kommuniziert per JSON-RPC über stdin/stdout und verwendet für jede Analyse exakt dieselbe Pipeline wie der Interpreter. Damit sind Diagnosen im Editor und auf der Kommandozeile garantiert identisch.

5.1 Architektur

Klasse	Verantwortung
MiniCpplLanguageServerMain	Start über stdio, Verbindung mit dem Client
MiniCpplLanguageServer	Lebenszyklus (initialize, shutdown, exit), Capabilities
MiniCpplTextDocumentService	Dokumentsynchronisation, entprellte Diagnosen, Verteilung der Anfragen
Document, Analysis	offenes Dokument; unveränderlicher Analyse-Snapshot einer Version
SymbolIndex	jedes Vorkommen eines Bezeichners mit der Deklaration, auf die es verweist
AstNodes, Names	AST-Traversierung, Gültigkeitsbereich an einer Position, Signaturen
Hovers, Completions, Navigation, CodeFormatter, CodeActions	die einzelnen Features
SourceText, Tokens, Positions	Offsets ↔ Positionen (Interpreter 1-basiert, LSP 0-basiert), Token-Suche, Diagnose-Umrechnung

Tabelle 8: Klassen des Language-Servers

Beim initialize meldet der Server folgende Fähigkeiten: inkrementelle Textsynchronisation, Hover, Completion mit den Triggerzeichen `.` und `>`, Definition, Referenzen, Document Highlight, Document Symbol, Rename mit prepareRename, Formatierung und Code-Actions der Art quickfix.

5.2 Synchronisation und Entprellung

Der Client überträgt Änderungen inkrementell als Bereichsänderungen, die `Document.update` der Reihe nach auf den Text anwendet. Jede Änderung plant auf einem einzelnen Analyse-Thread eine Neuanalyse mit 200 ms Verzögerung und bricht eine noch ausstehende ab. Beim Tippen wird so nur einmal nach der letzten Änderung analysiert. Ergebnisse werden nur veröffentlicht, wenn die analysierte Version noch aktuell ist.

Anfragen wie Hover oder Completion warten nicht auf den Timer: `Document.analysis()` berechnet die Analyse sofort, falls für die aktuelle Version noch keine vorliegt. Eine Anfrage sieht dadurch nie einen veralteten Stand.

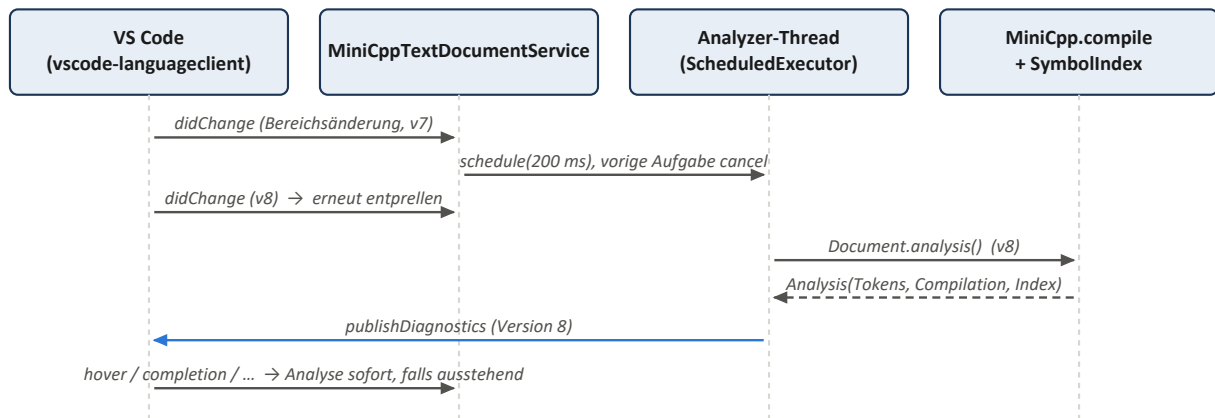


Abbildung 4: Ablauf von Änderungen, Entprellung und Anfragen

Hinweis: Die Synchronisation ist inkrementell, die Analyse selbst aber nicht: Nach jeder (entprellten) Änderung läuft die vollständige Pipeline über das gesamte Dokument. Bei den gemessenen Frontend-Zeiten (Abschnitt 7.3: ca. 3 ms für 100 Klassen) ist das für realistische Dateigrößen unkritisch. Im Arbeitsplan war ursprünglich inkrementelles *Parse*n vorgesehen – im Bericht sollte die Abweichung begründet werden.

5.3 Analyse-Snapshot und Fehlertoleranz

Eine *Analysis* bündelt für eine Dokumentversion den Quelltext, den Token-Strom, die *Compilation* und den Symbolindex. Zwei Mechanismen machen den Server robust gegenüber unfertigem Code:

- **Letzte parsebare Version.** Während des Tippens ist Code oft syntaktisch unvollständig – nach *konto.* existiert kein AST. Jede Analyse verweist daher über *lastParsed* auf die jüngste Version mit AST. Die *Completion* ermittelt Typ und Member des Ausdrucks vor dem Punkt aus dieser Version, und zwar auch über Ketten wie *a.b->c()*..
- **Absturzisolation.** Ausnahmen in Pipeline oder Indexierung werden abgefangen und protokolliert; der Server liefert dann leere Ergebnisse, statt die Verbindung zu verlieren.

5.4 Symbolindex

AST-Knoten kennen ihren Quellbereich, aber nicht die exakte Position ihres *Namens*. Der *SymbolIndex* kombiniert deshalb den aufgelösten AST mit dem Token-Strom und legt für jedes Vorkommen eines Bezeichners einen Eintrag *Occurrence(start, end, symbol, target, declaration)* in einer nach Offset sortierten *TreeMap* ab. Eine Positionsanfrage ist damit ein *floorEntry*-Aufruf in $O(\log n)$.

Die Unterscheidung zwischen *target* und *symbol* ist der Kern der Navigationsfeatures: *target* ist die exakte Deklaration (z. B. die gewählte Überladung oder der aufgerufene Konstruktor) und dient *Go-to-Definition* und *Hover*. *symbol* gruppiert alle Vorkommen, die gemeinsam umbenannt werden müssen: Konstruktoren gehören zu ihrer Klasse, überschreibende Methoden zur überschriebenen Basismethode.

5.5 Features

Feature	Verhalten
publishDiagnostics	Fehler aller statischen Phasen (Syntax, Namen, Typen) mit exaktem Bereich
hover	Deklaration als C++-Signatur mit Art (<code>local variable</code> , <code>virtual method</code> , <code>overrides B::f ...</code>); an Operatoren und Literalen der statische Typ des Ausdrucks
completion	Variablen und Parameter im Gültigkeitsbereich, Member der eigenen Klasse, Funktionen, Klassen, Schlüsselwörter, Typen; nach <code>./-></code> die Member (inkl. geerbter) des Objekts
definition	gewählte Überladung, statisch gebundene Methode, Konstruktor bei <code>A(...)</code> / <code>new A</code>
references, documentHighlight, rename	Klassen samt Konstruktornamen und Typverwendungen; überschreibende Methoden gemeinsam mit der Basismethode; Built-ins und Keywords sind nicht umbenennbar
documentSymbol	Outline mit Klassen, Feldern, Konstruktoren, Methoden und Funktionen
formatting	auf Basis des Parse-Trees: K&R-Klammern, <code>public</code> : auf Klassenebene, <code>T* p</code> , Leerzeichen um binäre Operatoren; Kommentare und einzelne Leerzeilen bleiben erhalten
codeAction	Quick Fixes: fehlendes Token einfügen, überzähliges entfernen, ähnlichen Namen vorschlagen, <code>.</code> → <code>-></code> , <code>()</code> an Methodennamen, leeres <code>main</code> ergänzen

Tabelle 9: Umgesetzte LSP-Features

TODO (Entwurf): Screenshots der Features in VS Code (Hover, Completion nach `->`, Quick Fix, Rename) einfügen.

5.6 VS-Code-Extension

Die Extension (`vscode/`) registriert die Sprache `minicpp` für `*.mcpp`-Dateien, liefert eine TextMate-Grammatik für Syntaxhervorhebung sowie eine Sprachkonfiguration (Kommentare, Klammerpaare) und startet den Server über `vscode-languageclient`. Der Server wird in folgender Reihenfolge gesucht: konfigurierter Pfad (`minicpp.server.path`), im `.vsix`-Paket gebündelter Server (direkt mit `java` gestartet, da Startskripte im Paket ihr Ausführungsrecht verlieren), zuletzt der im Repository gebaute Server. Das Java-Programm lässt sich mit `minicpp.java.path` festlegen, `minicpp.trace.server` protokolliert den JSON-RPC-Verkehr.

6 GenAI-Assistenzserver

Der Assistenzserver (Modul `mcp`) stellt KI-gestützte Code-Assistenz für MiniC++ als HTTP/JSON-Dienst bereit. Er nutzt ein lokal laufendes Sprachmodell über Ollama und verbindet dessen Antworten mit dem echten MiniC++-Compiler.

TODO (Entwurf): Das Modul heißt `mcp`, implementiert aber eine REST-Schnittstelle und nicht das *Model Context Protocol* (JSON-RPC mit `Tools/Resources`). Für die Endfassung klären: entweder Begriff im Bericht präzisieren („GenAI-REST-Server“) oder eine MCP-Schnittstelle ergänzen.

6.1 Endpunkte

Endpunkt	Anfrage (JSON)	Antwort
POST /complete	code, line (1-basiert), column (0-basiert)	completion: einzufügender Text
POST /explain	code	explanation
POST /refactor	code, optional instruction	code, rationale, compiles, diagnostics
POST /detect-bugs	code	compilerDiagnostics, findings[{line, description}], analysis
GET /health	–	Status, Provider, Modell, Erreichbarkeit

Tabelle 10: Endpunkte des Assistenzservers

Der Server basiert auf dem im JDK enthaltenen `com.sun.net.httpserver.HttpServer` mit einem Pool von vier Threads und Gson für JSON; weitere Abhängigkeiten gibt es nicht. Er bindet standardmäßig nur an `127.0.0.1:8080`. Fehler werden einheitlich als `{"error": ...}` gemeldet: HTTP 400 bei ungültiger Anfrage, 404 bei unbekanntem Pfad, 405 bei falscher Methode, 413 bei Anfragen über 1 MiB und 502, wenn das Modell nicht erreichbar ist.

6.2 Provider-Abstraktion

Die Schnittstelle `GenAiProvider` kapselt das Sprachmodell (`generate(system, prompt)`, `isAvailable()`). Der `OllamaProvider` ruft `/api/generate` ohne Streaming und mit Temperatur 0,2 auf – Code-Assistenz soll fokussiert statt kreativ sein. Modell (Standard `codellama`), URL und Timeout (120 s) sind per Kommandozeile oder Umgebungsvariable konfigurierbar. Der `MockProvider` implementiert dieselbe Schnittstelle deterministisch; er ermöglicht Tests des gesamten Servers ohne laufendes Modell und einen Betrieb mit `--provider mock`.

6.3 Prompt-Design

Sprachmodelle kennen C++, aber nicht MiniC++. Jeder System-Prompt enthält deshalb eine kompakte Sprachbeschreibung, die neben den Features ausdrücklich aufzählt, was fehlt (Arrays, Casts, ++, +=, this, Standardbibliothek ...). Ohne diese Negativliste schlagen Modelle bevorzugt idiomatisches C++ vor, das MiniC++ nicht akzeptiert. Die endpunktspezifischen Anweisungen erzwingen ein maschinenlesbares Antwortformat:

- `/complete` markiert die Cursorposition mit `<CURSOR>` und verlangt ausschließlich den einzufügenden Text; umschließende Markdown-Codeblöcke werden trotzdem defensiv entfernt.
- `/refactor` verlangt den vollständigen Code in genau einem ````cpp`-Block, gefolgt von einer Zeile `Rationale:`.
- `/detect-bugs` übergibt den Code mit Zeilennummern und verlangt Befunde im Format `LINE <n>: <text>` oder `NONE`; ein regulärer Ausdruck überführt sie in strukturierte `findings`.

6.4 Absicherung durch den Compiler

Das wichtigste Qualitätsmerkmal des Servers ist, dass er Modellantworten nicht ungeprüft weitergibt:

- **Grounding:** Für `/explain` und `/detect-bugs` werden die Diagnosen des MiniC++-Compilers an den Prompt angehängt. Das Modell muss Syntax- und Typfehler nicht raten, sondern kann sie als gesichert behandeln.
- **Verifikation:** Jeder Refactoring-Vorschlag wird vor der Rückgabe kompiliert; das Feld `compiles` und die Diagnosen zeigen dem Aufrufer, ob der Vorschlag gültiges MiniC++ ist.

- **Strukturierte Ergänzung:** `/detect-bugs` liefert die Compilerdiagnosen zusätzlich getrennt von den Modellbefunden, sodass ein Client sichere und heuristische Befunde unterschiedlich darstellen kann.

Code ohne `main` wird dabei nur als Deklarationssammlung geprüft, damit auch einzelne Funktionen oder Klassen analysiert werden können.

TODO (Entwurf): Qualitative Evaluation der Modellantworten ergänzen (z. B. Anteil kompilierender Refactorings für `codellama` vs. `deepseek-coder` auf den Testprogrammen, typische Fehlermuster, Antwortzeiten).

7 Qualitätssicherung und Evaluation

7.1 Teststrategie

Alle 218 automatisierten Tests laufen mit `./gradlew build` bzw. `./gradlew test` und waren zum Redaktionsschluss grün. Sie gliedern sich wie folgt:

Testklasse	Modul	Tests	Gegenstand
ErrorTest	interpreter	129	ungültige Programme: erwartete Phase und Meldung des ersten Fehlers, inkl. Laufzeitfehler
CppExporterTest	interpreter	14	Übersetzung nach C++
ReplTest	interpreter	13	Sitzungen, Fortsetzungszeilen, Rücksetzen bei Fehlern, Befehle
ParserTest	interpreter	10	Präzedenz, Literale und Escapes, Kurzformen, Referenztypen, Klassen
ProgramTest	interpreter	9	Golden-File-Tests der Beispielprogramme
FeaturesTest	lsp	19	Hover, Completion, Navigation, Rename, Formatierung, Quick Fixes
DiagnosticsTest	lsp	3	Diagnosen Ende-zu-Ende mit Client-Stub: Positionen, Syntaxfehler, Löschen beim Schließen
AssistantServiceTest	mcp	9	Prompts, Antwortverarbeitung, Compiler-Absicherung (Mock)
McpServerTest	mcp	8	HTTP-Ebene, Fehlercodes
OllamaProviderTest	mcp	4	Ollama-Protokoll gegen lokalen Test-Server
Summe		218	

Tabelle 11: Automatisierte Tests (Stand 27.09.2026)

Golden-File-Tests

Das Verzeichnis `interpreter/src/test/resources/programs` enthält neun thematische Programme (Grundlagen, Kontrollfluss, Funktionen, Klassen, Vererbung, Zeiger, Scoping, `void main`, Exit-Codes), jeweils mit einer `.expected`-Datei der erwarteten Ausgabe. Ein Kommentar `// expect-exit: N` legt einen abweichenden Exit-Code fest. Dieselben Dateien dienen auch dem GCC-Vergleich.

7.2 Differenzieller Vergleich mit GCC

Die stärkste Korrektheitsaussage liefert der Vergleich mit einem echten C++-Compiler. Das Skript `scripts/compare-gcc.sh` führt jedes Testprogramm (1) im Interpreter aus, (2) übersetzt es mit `minicpp to-cpp` nach C++, kompiliert es mit `g++ -std=c++17 -fwrapv -O1` und führt es aus, und vergleicht (3) beide Ausgaben und Exit-Codes mit der erwarteten Ausgabe. `-fwrapv` stellt sicher, dass GCC Ganzzahlüberlauf ebenfalls als Zweierkomplement behandelt.

Continuous Integration

Die GitHub-Actions-Pipeline (`.github/workflows/ci.yml`) läuft bei jedem Push und Pull-Request auf `ubuntu-latest`: Build und Tests aller Module mit Temurin 21, Build der VS-Code-Extension mit Node 22 und abschließend der GCC-Vergleich.

TODO (Entwurf): Aktuelles Ergebnis des GCC-Vergleichs aus der CI (Anzahl PASS/FAIL) eintragen; lokal ist kein g++ installiert.

7.3 Performance und Skalierbarkeit

Die Benchmarks im Modul `benchmark` verwenden JMH und messen die mittlere Laufzeit. `InterpreterBenchmark` misst reine Interpretation (übersetzt wird einmal pro Trial) für vier Lasttypen mit Größenparameter n ; `FrontendBenchmark` misst Parsen bzw. Parsen und Prüfen generierter Programme mit 10, 100 und 1 000 Klassen.

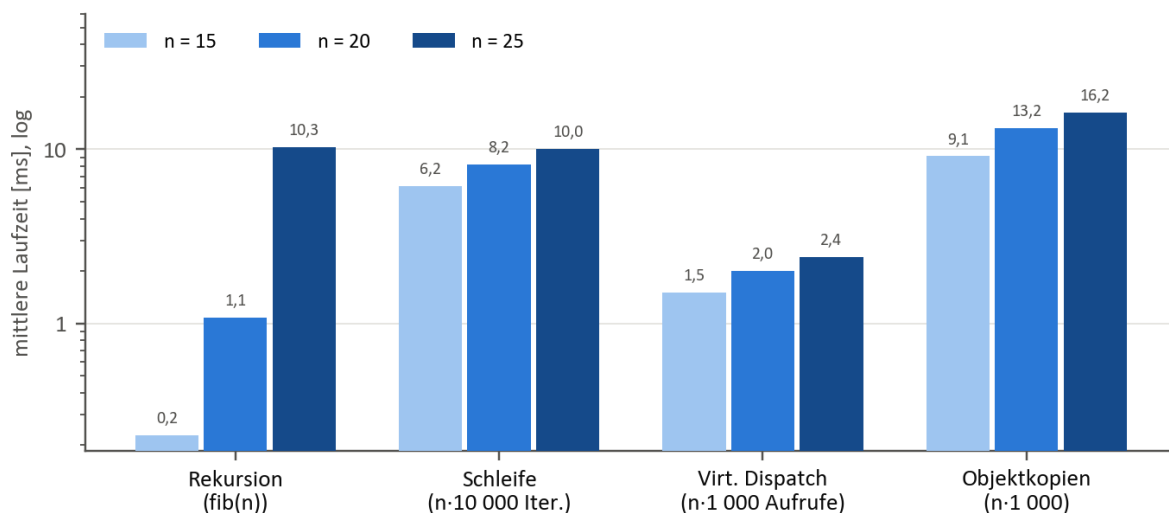


Abbildung 5: Laufzeit der Interpreter-Benchmarks nach Lasttyp und Größe n (logarithmische Achse)

Die Rekursion `fib(n)` wächst erwartungsgemäß exponentiell (Faktor $\approx 1,6$ pro Schritt von n): von 0,23 ms bei $n = 15$ auf 10,3 ms bei $n = 25$, bei rund 243 000 Aufrufen also etwa 40 ns pro interpretiertem Aufruf. Die drei anderen Lasten wachsen linear in n ; sie zeigen jeweils einen konstanten Sockel, der auf Thread-Start (512-MiB-Stack) und JIT-Aufwärmeeffekte zurückgeht. Virtueller Dispatch ist mit 2,41 ms für 25 000 Aufrufe günstig, da `vtables` pro Klasse nur einmal aufgebaut werden. Objektkopien sind die teuerste Lastart, weil jede Kopie neue Cells und Maps alloziert.

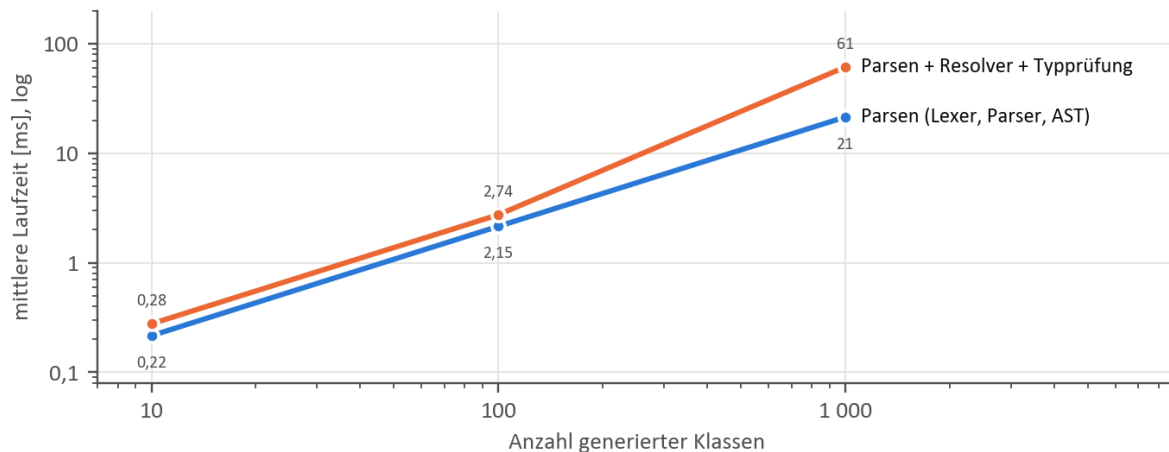


Abbildung 6: Skalierung von Parsen und statischer Analyse mit der Programmgröße (log-log)

Das Frontend skaliert zwischen 10 und 100 Klassen linear (Faktor 10 bei zehnfacher Größe). Für 1 000 Klassen benötigt das Parsen 21,4 ms, Parsen und Prüfen 61,2 ms; der überproportionale Anstieg der Prüfung in diesem Punkt ist auffällig, hat aber in diesem kurzen Messlauf auch die größte Streuung ($\pm 47,5$ ms) und muss mit längeren Läufen bestätigt werden.

Messumgebung: JMH mit 1 Fork, 2×1 s Warm-up und 3×1 s Messung; AMD Ryzen 7 9800X3D, OpenJDK 64-Bit Server VM 25.0.1+8-LTS. Die Rohdaten liegen in `benchmark/build/jmh-result.json`.

TODO (Entwurf): Die Werte stammen aus einem kurzen Vorab-Lauf mit großer Streuung. Für die Endfassung mit den Standardeinstellungen oder mehr Forks messen (`./gradlew :benchmark:jmh -Pjmh="-f 3 -wi 5 -i 10"`) und ggf. einen Vergleich mit `g++ -O0` ergänzen.

7.4 Bewertung

Die Tests decken jede Phase einzeln sowie das Zusammenspiel über Golden Files ab; der GCC-Vergleich belegt, dass die Semantik nicht nur mit den eigenen Erwartungen, sondern mit der Referenzimplementierung übereinstimmt. Besonders wertvoll war die Kombination aus ErrorTest und Laufzeit-Lebensdauerprüfung: Jeder Fall von undefiniertem Verhalten in C++ ist als eigener Testfall mit erwarteter Meldung festgehalten. Die Performance ist für den Einsatzzweck – Lehre, interaktive Nutzung, Editor-Analyse – ausreichend: typische Beispielprogramme laufen im einstelligen Millisekundenbereich, und die statische Analyse einer Datei mit 100 Klassen bleibt deutlich unter der 200-ms-Entprellzeit des Language-Servers.

8 Projektorganisation

8.1 Arbeitsplan

Das Projekt war auf zwölf Wochen angelegt. Die Kernphasen wurden gemeinsam bearbeitet, die Erweiterungen (LSP und GenAI) parallel von jeweils einem Teammitglied.

Woche	Schwerpunkt	Verantw.
1	Repository (Gradle), ANTLR4 einrichten, Grammatik-Entwurf, CI	C + D
2–4	ANTLR4-Lexer und -Parser, AST-Klassenhierarchie, Fehlerbehandlung	C + D
5–6	Two-Pass-Resolver, Typprüfung, semantische Prüfungen	C + D
7–8	Tree-Walking-Interpreter, REPL, Objektmodell, vtables	C + D
9–11	LSP4J-Server, VS-Code-Extension	C
9–11	GenAI-Server, Ollama-Anbindung, Mock-Provider	D
12	Benchmarks, Korrektheitstests, Dokumentation, Walk-Through	C + D

Tabelle 12: Arbeitsplan (C = Clemens Vogtländer, D = Dennis Gorpinic)

8.2 Aufgabenteilung

Aufgabe	C. Vogtländer	D. Gorpinic
Grammatik (Lexer und Parser), AST-Hierarchie	50 %	50 %
Resolver, Typprüfung und Überladungsauflösung	50 %	50 %
Interpreter, Objektmodell (vtable, Slicing), REPL	50 %	50 %
LSP-Server und VS-Code-Extension	100 %	–
GenAI-Server, Ollama-Anbindung, Prompt-Engineering	–	100 %
Mock-Provider für Tests	20 %	80 %
Korrektheitstests, GCC-Vergleich, JMH-Benchmarks	50 %	50 %
Abschlussdokumentation und Walk-Through	50 %	50 %

Tabelle 13: Aufgabenteilung

Entwickelt wurde in einem gemeinsamen Git-Repository; die Implementierung des Interpreters liegt auf dem Branch `interpreter`. Die CI prüft jeden Push, sodass Regressionen im gemeinsam genutzten Kern früh sichtbar werden – wichtig, weil LSP- und GenAI-Server direkt von dessen AST-Annotationen abhängen.

TODO (Entwurf): Tatsächlichen Verlauf gegenüber dem Plan reflektieren: Was hat länger gedauert (z. B. Zeiger-Semantik, die nachträglich in den Umfang aufgenommen wurde), welche Entscheidungen wurden revidiert, was würde das Team anders machen?

9 Grenzen und Ausblick

9.1 Bekannte Grenzen

- **Kein AST bei Syntaxfehlern.** Die Pipeline bricht nach Syntaxfehlern ab. Der Language-Server überbrückt dies für die Completion mit der letzten parsebaren Version; Hover und Navigation sind in einer syntaktisch fehlerhaften Datei jedoch nicht verfügbar. Eine Fehlerrecovery mit ErrorExpr-Knoten würde partielle ASTs ermöglichen.
- **Erste Fehlerphase gewinnt.** Meldet der Resolver Fehler, entfällt die Typprüfung; Typfehler werden erst nach Behebung der Namensfehler sichtbar.
- **Vollständige Neuanalyse.** Jede Änderung analysiert das gesamte Dokument neu (siehe Abschnitt 5.2).
- **Interpretationsgeschwindigkeit.** Der Tree-Walker schlägt Variablen über eine IdentityHashMap pro Frame nach und alloziert für jeden Wert eine Cell. Für den Lehrkontext genügt das; für rechenintensive Programme wäre er um Größenordnungen langsamer als kompilierter Code.
- **Ein Dokument pro Programm.** MiniC++ kennt keine Module; der Language-Server behandelt jede Datei unabhängig.

9.2 Mögliche Weiterentwicklungen

- **Slot-Auflösung:** Der Resolver könnte jeder lokalen Variable einen Index im Frame zuweisen, sodass Zugriffe zu Array-Operationen werden – der naheliegendste Performancegewinn.
- **Bytecode oder Truffle:** Eine Übersetzung in einen kompakten Bytecode oder eine Implementierung auf GraalVM Truffle würde JIT-Kompilierung ermöglichen.
- **Sprachumfang:** for, break/continue, const und Arrays sind die am häufigsten vermissten Konstrukte und ließen sich in die bestehende Architektur einfügen.
- **Debug Adapter Protocol:** Da der Interpreter jeden Knoten mit Quellbereich ausführt, liegt ein Debugger für VS Code (Breakpoints, Variablenansicht) nahe.
- **MCP-Anbindung:** Die Assistenzfunktionen zusätzlich als Model-Context-Protocol-Tools bereitstellen, sodass KI-Agenten MiniC++-Code prüfen und ausführen können.

9.3 Fazit

Das Projekt zeigt, dass eine klar abgegrenzte Sprache mit einem sauber geschichteten Frontend eine tragfähige Grundlage für ein ganzes Werkzeug-Ökosystem ist. Die Entscheidung, alle Analyseergebnisse als Annotationen am AST und alle Probleme als positionsgenaue Diagnostics zu modellieren, hat sich ausgezahlt: Interpreter, REPL, Language-Server und GenAI-Server nutzen dieselbe Pipeline und liefern konsistente Ergebnisse. Die Korrektheit gegenüber C++ ist durch den differenziellen GCC-Vergleich belegt, und die Laufzeitprüfungen machen undefiniertes Verhalten, das in C++ schwer zu finden ist, für Lernende sichtbar.

TODO (Entwurf): Fazit nach Abschluss der Evaluation überarbeiten; ggf. Abschnitt zu Lessons Learned ergänzen.

Anhang

A Build und Nutzung

Voraussetzung ist ein JDK 21 oder neuer; für die Extension zusätzlich Node.js.

```
./gradlew build                # alle Module bauen und testen
./gradlew :interpreter:installDist # -> interpreter/build/install/minicpp/bin/minicpp
./gradlew :lsp:installDist      # -> lsp/build/install/minicpp-lsp/bin/minicpp-lsp
./gradlew :mcp:run --args="--provider mock" # GenAI-Server ohne Ollama auf :8080
./gradlew :benchmark:jmh        # Benchmarks, Ergebnis in benchmark/build/jmh-result.json
cd vscode && npm install && npm run package # -> minicpp-1.0.0.vsix
```

Listing 8: Build-Befehle

Befehl	Wirkung
minicpp run datei.cpp	Programm ausführen; Exit-Code = Rückgabewert von main
minicpp check datei.cpp	nur Syntax-, Namens- und Typprüfung
minicpp repl [datei.cpp]	REPL, optional mit vorher geladener Datei
minicpp ast datei.cpp	AST ausgeben
minicpp to-cpp datei.cpp	nach Standard-C++17 übersetzen

Tabelle 14: Unterbefehle der Kommandozeile

Option	Umgebungsvariable	Standard
--host	MCP_HOST	127.0.0.1
--port	MCP_PORT	8080
--provider	MCP_PROVIDER	ollama (alternativ mock)
--ollama-url	OLLAMA_URL	lokale Ollama-Instanz
--model	OLLAMA_MODEL	codellama
--timeout	OLLAMA_TIMEOUT	120 s

Tabelle 15: Konfiguration des GenAI-Servers

B Grammatik: Deklarationen und Anweisungen

```
program      : declaration* EOF ;
replInput    : (declaration | statement)* expr? EOF ;
declaration  : functionDef | classDef ;
classDef     : CLASS Identifier (COLON PUBLIC Identifier)?
              LBRACE PUBLIC COLON memberDecl* RBRACE SEMI ;
memberDecl   : fieldDecl | constructorDef | methodDef ;
fieldDecl    : type Identifier SEMI ;
constructorDef: Identifier LPAREN paramList? RPAREN block ;
methodDef    : VIRTUAL? type Identifier LPAREN paramList? RPAREN block ;
functionDef  : type Identifier LPAREN paramList? RPAREN block ;
param        : (type | typeRef) Identifier ;
typeRef      : type AMP ;
varDecl      : type Identifier
              | type Identifier LPAREN argList RPAREN
              | (type | typeRef) Identifier ASSIGN expr ;
ifStmt       : IF LPAREN expr RPAREN statement (ELSE statement)? ;
whileStmt    : WHILE LPAREN expr RPAREN statement ;
returnStmt   : RETURN expr? ;
deleteStmt   : DELETE expr ;
newExpr      : NEW (baseType | Identifier) (LPAREN argList? RPAREN)? ;
```

Listing 9: Parser-Regeln für Deklarationen und Anweisungen (vollständig in MiniCpp.g4)

C Abkürzungen

Abkürzung	Bedeutung
ADR	Architecture Decision Record
AST	Abstract Syntax Tree, abstrakter Syntaxbaum
CI	Continuous Integration
JMH	Java Microbenchmark Harness
LSP	Language Server Protocol
MCP	Model Context Protocol
REPL	Read-Eval-Print-Loop
vtable	Tabelle virtueller Methoden einer Klasse